Why Django is so popular

Django is popular for many reasons but the largest being that it is a “batteries included” framework. It comes with built in tools that most applications that are being developed need like auth, forms, routing, ORM etc. It says that it also emphasises security as well with stron defaults so thats nice too. It also has a massive community and support team which makes asking for help nice.

Five large companies/products that use Django

1. **Instagram**— Social platform. Instagram has used Django at massive scale on the backend (they’ve described it as their Django deployment).

2. **Disqus** — Comment hosting and a discussion platform embedded on other applications and platforms. Disqus uses Django for the majority of its web traffic and talks about their scaling Django for very high request volume.

3. **The Washington Post** — obviously THE News and media publisher. Django has been used to power parts of washingtonpost.com (example: their Congress votes database application).

4. **Mozilla)** — Browser add-ons marketplace for Firefox (little outdated i know). Mozilla maintains the Addons Server, described as the “addons.mozilla.org Django app and API.”

5. **Prezi** — Presentation and visual communication platform. Prezi engineering has described internal infrastructure built in Python using the Django web framework.

Django “Would you use it?” scenarios (explain why/why not)

1. **You need to develop a web application with multiple users.**
   Yes. Django is a goodchoice because it has built-in authentication/authorization patterns, sessions, and a structured approach to building multi-user apps with database-backed models.

2. **You need fast deployment and the ability to make changes as you proceed.**
   Yes. Django supports much faster development thanks to reusable apps, the ORM + migrations, and built-in admin tools that speed up iteration and updates.

3. **You need to build a very basic application with no database access or file operations.**
   Probably not. Django can do it, but it’s usually huge amounts of overkill if you don’t

need the database/ORM/admin features—something lighter (even a simple script) will be a better fit.

4. **You want to build an application from scratch and want a lot of control over how it works.**
   Usually no (or "only if you accept Django's conventions"). Django gives you a lot of structure and helpful defaults; if your main goal is maximum low-level control over architecture, Django's opinionated design may feel restrictive.

5. **You're about to start working on a big project and are afraid of getting stuck and needing additional support.**
   Yes. Django has a huge community, lots of documentation, and common solutions already documented, which lowers the risk of getting blocked.